Modbus RTU Relay 32CH

From Waveshare Wiki
Jump to: navigation, search

Overview

Modbus RTU Relay 32CH



(https://www.waveshare.com/modbus-rtu-relay-32ch.htm)

RS485

Electrical and Relay Safety Instructions

- This product must be operated by professional electricians or qualified personnel. During use, ensure electrical safety, leakage protection, and proper insulation.
- Before installing, maintaining, or replacing the relay device, always turn off the power and unplug the device.
- Do not attempt to disassemble the relay device to avoid damage or the risk of electric shock.
- Properly install and place the relay device. Do not use it in humid, overheated, flammable, or explosive environments to prevent accidents caused by improper installation or use.

1. Load Matching

- Ensure the relay's rated voltage and current match the load. Do not exceed the rated capacity.
- For inductive loads (motors, coils, lamps, etc.), the starting current may be much higher than the rated current. Choose a relay with sufficient current margin.

2. Short Circuit and Overcurrent Protection

- Install a **fuse** or **circuit breaker** in the relay circuit to prevent damage due to short circuits or accidental overcurrent.
- Ensure the load circuit has no short circuits during wiring, and select protection components with appropriate current ratings if necessary.

3. Arc and Switching Protection

- Relay switching generates arcs, which can cause contact wear or welding.
- For inductive loads, it is recommended to use **RC snubber circuits** or **varistors** for arc suppression.

4. Installation Environment

- Do not use the relay in humid, high-temperature, flammable, explosive, or dusty environments.
- Install the relay securely to avoid vibrations or shocks that may cause misoperation or damage.

5. Power-Off Operation

- Always cut off power before maintenance, wiring, or replacing the relay to ensure personnel and device safety.
- Latching relays are only powered when changing state. Avoid strong vibrations or strong magnetic fields while the relay is unpowered.

6. Status Confirmation

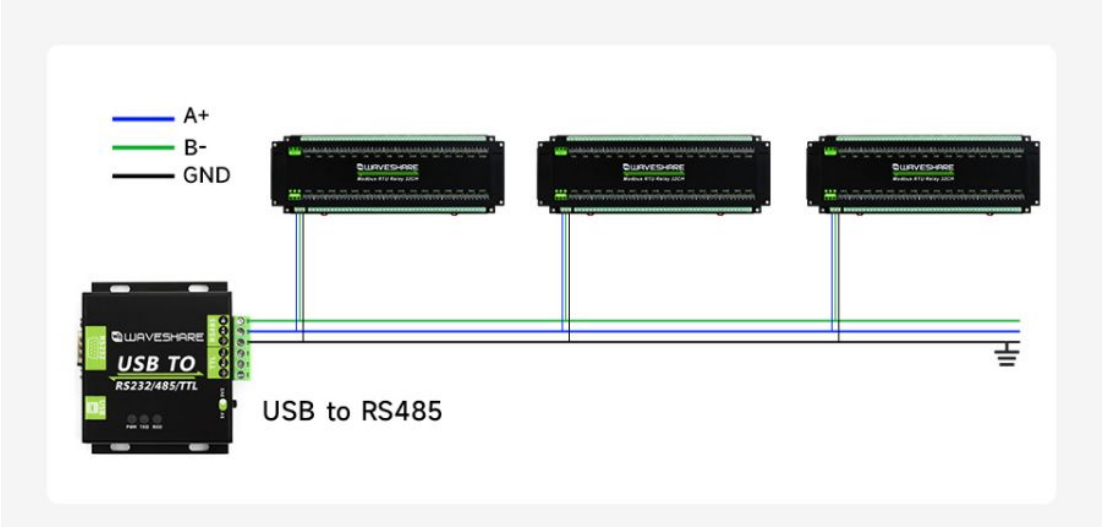
- After powering on, confirm or reset the relay status as needed to prevent abnormal operation caused by transportation, installation, or external disturbances.
- Avoid power interruption during relay operation to prevent uncertain status or contact damage.

7. Regular Inspection

- Periodically inspect relay contacts, terminals, and insulation to ensure proper operation.
- If abnormal heating, odor, or burn marks are detected, immediately cut off power and replace the relay.

Hardware Connection

- Connect the USB TO 485 to the target boards via cables, A-A and B-B connected as shown below:



(/wiki/File:Modbus-RTU-Relay-32CH-details-7.jpg)

Example Demonstration

The demo shows how the following two software operate.

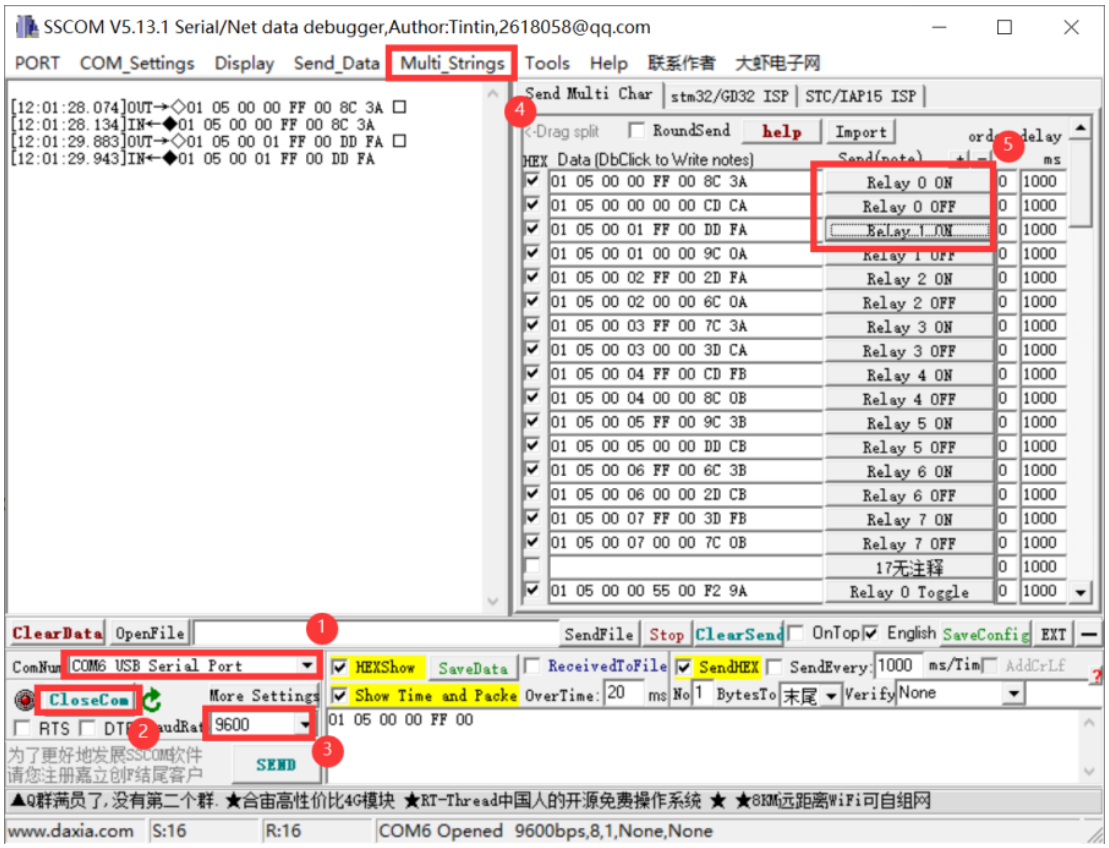
SSCOM serial port debugging assistant is more convenient to operate, free of installation, and more convenient for complete display and analysis of instructions, but the disadvantage is that the data is not intuitive.

Modbus Poll software is directly operated on the register, and the data display is more convenient to observe, but the disadvantage is that the instruction is not displayed completely, so you need to be familiar with the Modbus register operation.

You can test using any method. It is recommended to use the SSCOM serial port debugging assistant software for the first test.

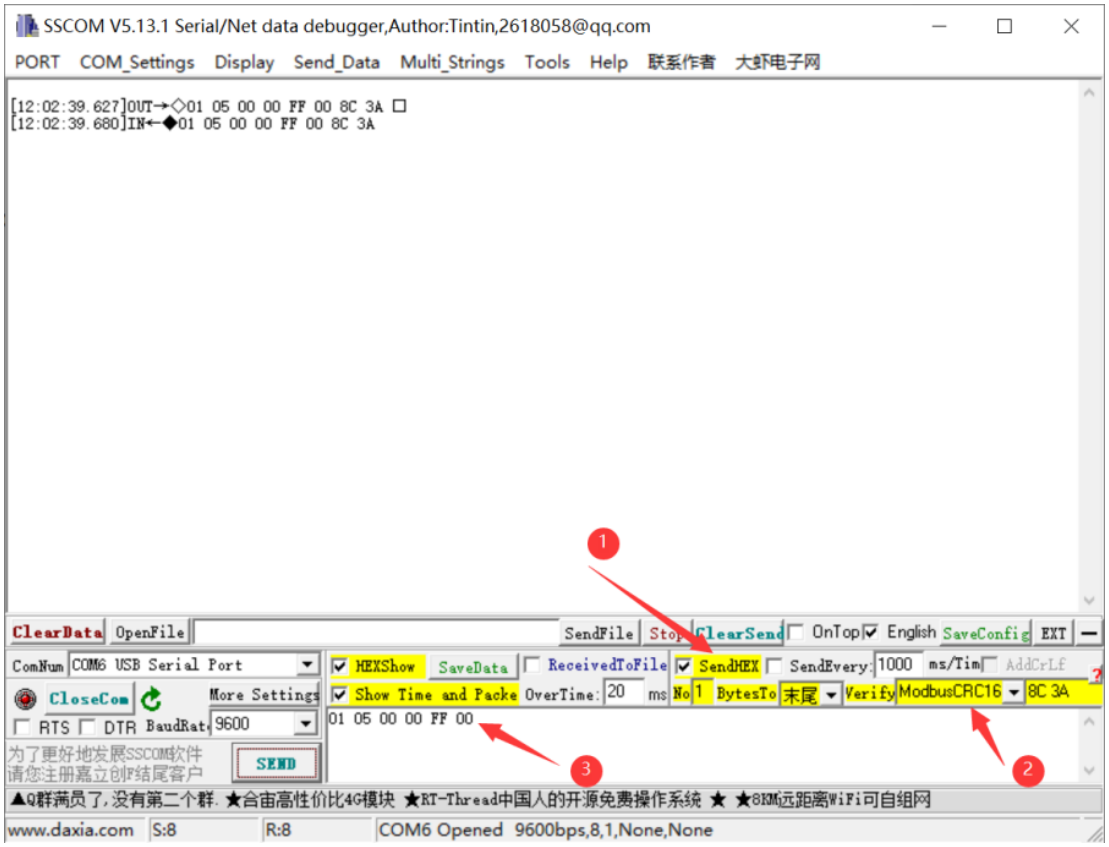
SSCOM Serial Port Debugging Assistant

- Download SSCOM Serial port debugging assistant and open it on the computer, open the corresponding port number, and set the baud rate as 9600. Click Multi-Char to open the Send Multi-Char window, and click the function to send the corresponding command.



(/wiki/File:Modbus-RTU-Relay-2-1.png)

- If you need to send other commands, choose SendHEX. For checksum validation, select ModbusCRC16. After entering the first six bytes of the command, clicking SEND will automatically add the CRC check code.

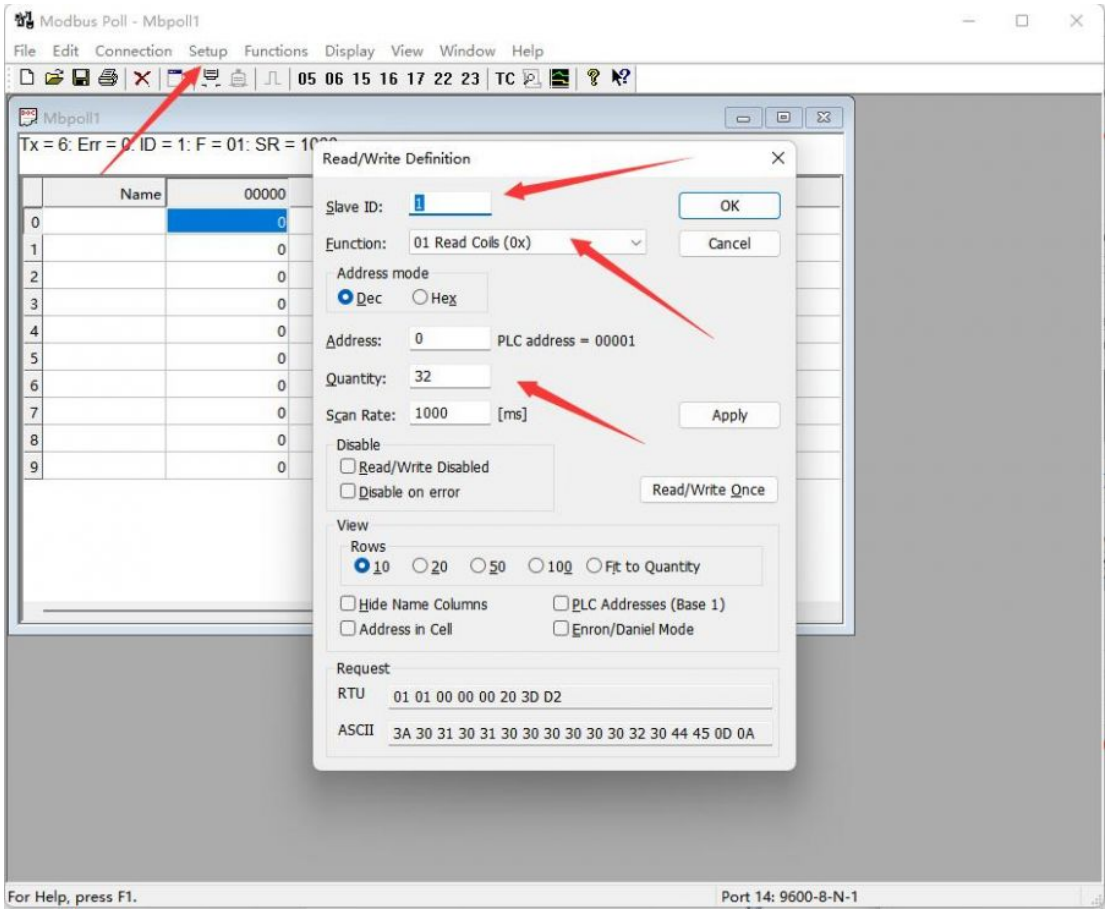


(/wiki/File:Modbus-RTU-Relay-3.png)

- For detailed control commands, please see the development protocol.

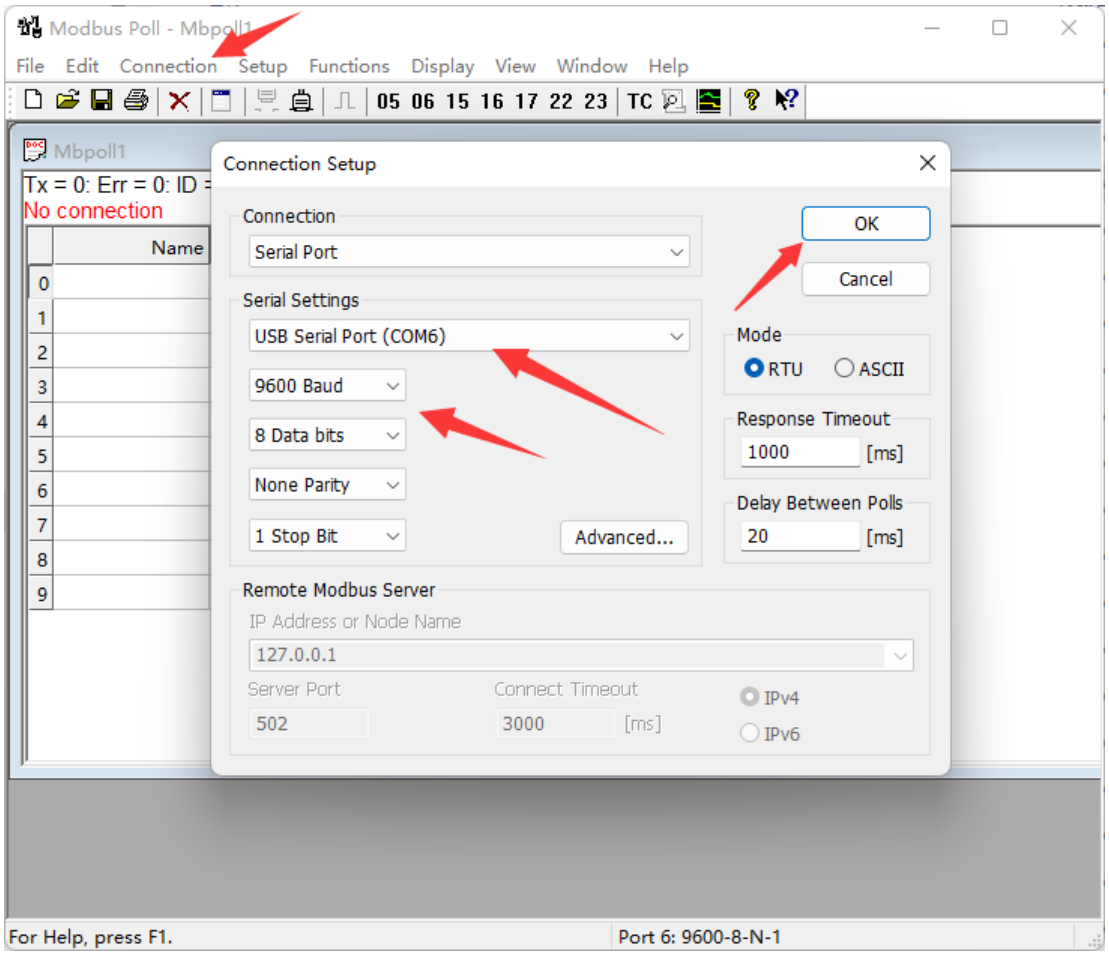
Modbus Poll Software

- It is not convenient to use the SSCOM software for observing the data, you can select Modbus Poll software (https://www.modbustools.com/download.html) to read the data. Download and install the Modbus Poll software.
- Open the software, select Setup->Read/Write Definition, select the actual device address for Slave ID, select 01 Read Coils function code for Function, and change Quantity to 32 channels. Click OK to confirm.



(/wiki/File:900px-Modbus-RTU-Relay-32CH-1.jpg)

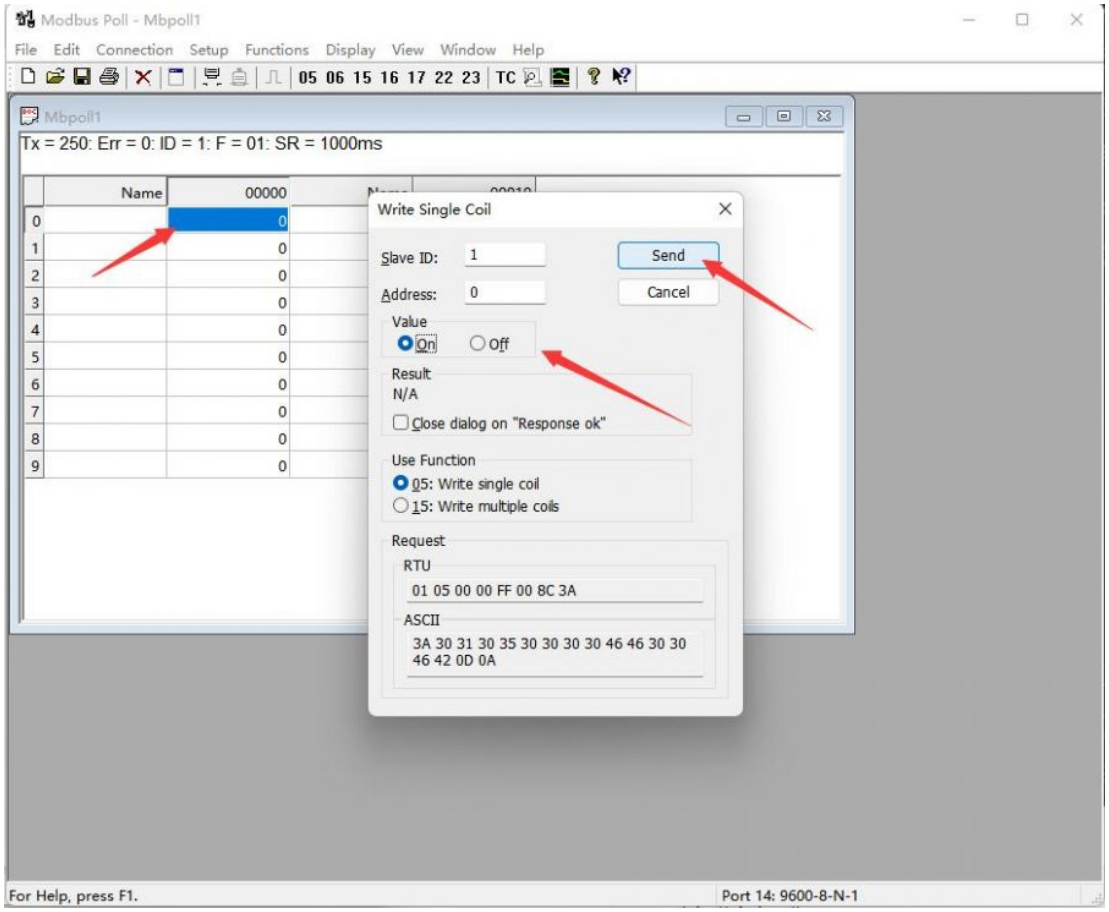
- Select Connection->Connect..., choose the corresponding serial port, set the baud rate to 9600, and select 8 Data bits and None Parity. Click OK to connect.



(/wiki/

File:Modbus-RTU-Analog-Input-3.png)

- After the connection is normal, you can check the current relay status. Select the corresponding channel, then double-click the status value to pop up the send page. Choose On or Off, then Click Send to control the relay opening and closing.



(/wiki/File:900px-Modbus-RTU-Relay-16CH-2.jpg)

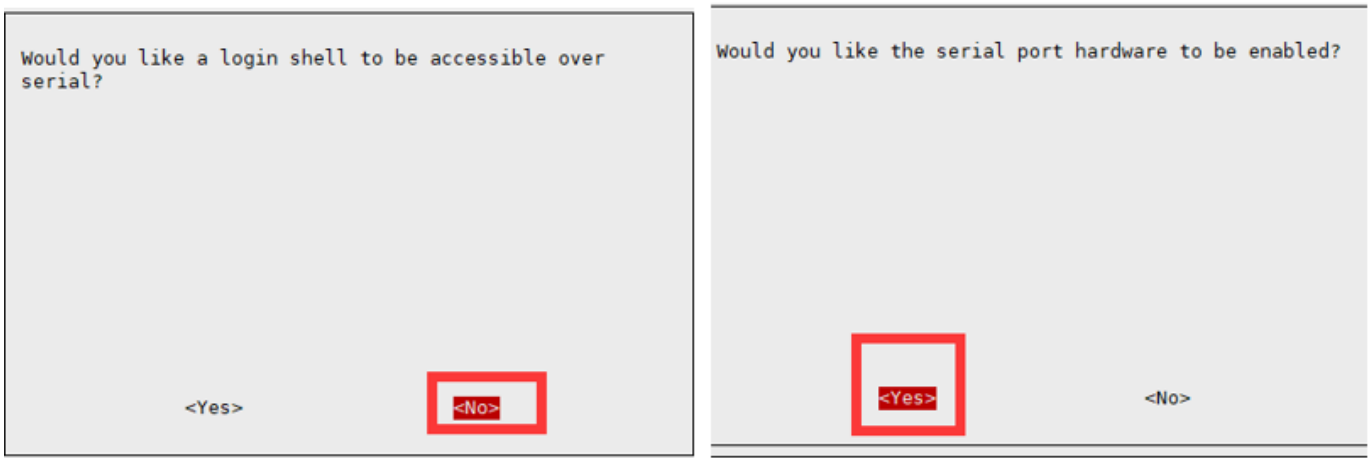
Demo Test

Note: RS485 can not be directly connected to the serial port of the Raspberry Pi, otherwise it may burn the device, you need to add 485 level conversion. For Raspberry Pi, it is recommended to work with the RS485 CAN HAT module. For NUCLEO-F103RB and Arduino, it is recommended to work with the RS485 CAN Shield module.

Raspberry Pi

Open the Raspberry Pi terminal and enter the following command to enter the configuration interface

```
sudo raspi-config
Select Interfacing Options -> Serial, disable shell access, and enable the hardwar
e serial port
```



(/wiki/File:L76X_GPS_Module_rpi_serial.png)

Then restart Raspberry Pi:

```
sudo reboot
```

Open the /boot/config.txt file, find the following configuration statement to enable the serial port, if not, you can add it to the end of the file.

```
enable_uart=1
```

For Raspberry Pi 3B users, the serial port is used for Bluetooth and needs to be commented out:

```
#dtoverlay=pi3-miniuart-bt
```

Then restart Raspberry Pi:

```
sudo reboot
```

Insert the RS485 CAN HAT into the Raspberry Pi, and connect the Modbus RTU Relay module to the RS485 CAN HAT through A and B.

If you are using other 485 devices, make sure to connect A-A, B-B.

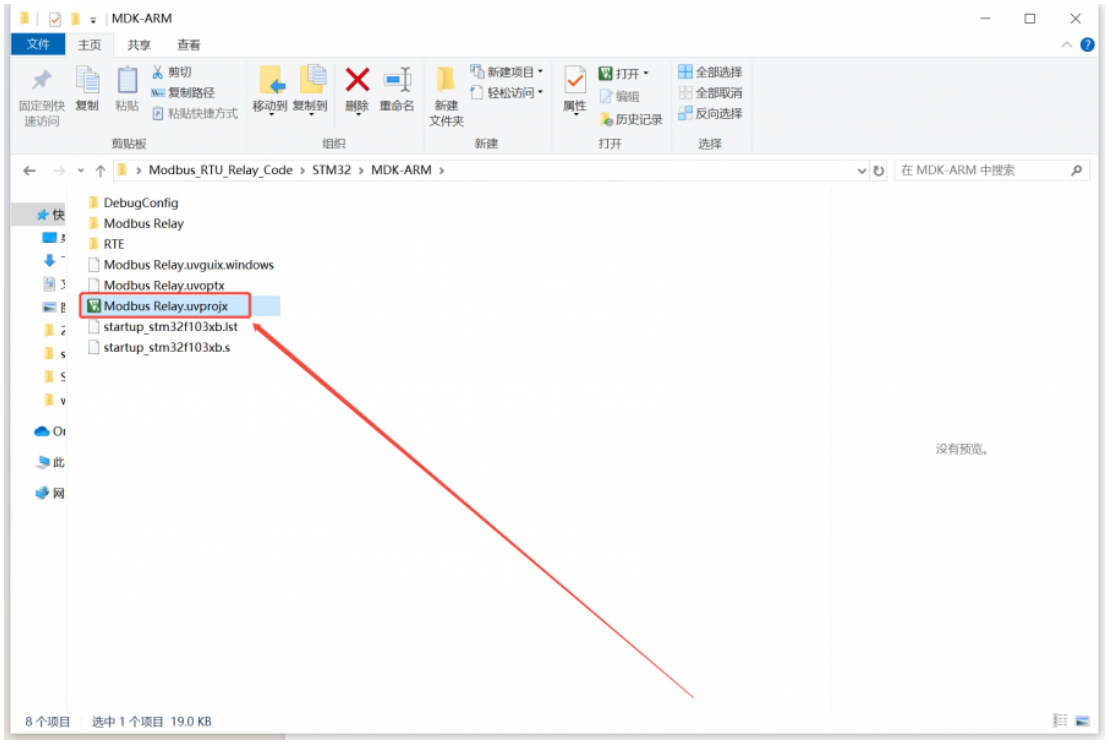
Run the following commands to run the demo:

```
sudo apt-get install unzip
wget https://files.waveshare.com/wiki/Modbus-RTU-Relay-32CH/Modbus_RTU_Relay_32CH_
Code.zip
unzip Modbus_RTU_Relay_32CH_Code.zip
cd Modbus_RTU_Relay_32CH_Code/Python3
sudo python3 main.py
```

STM32

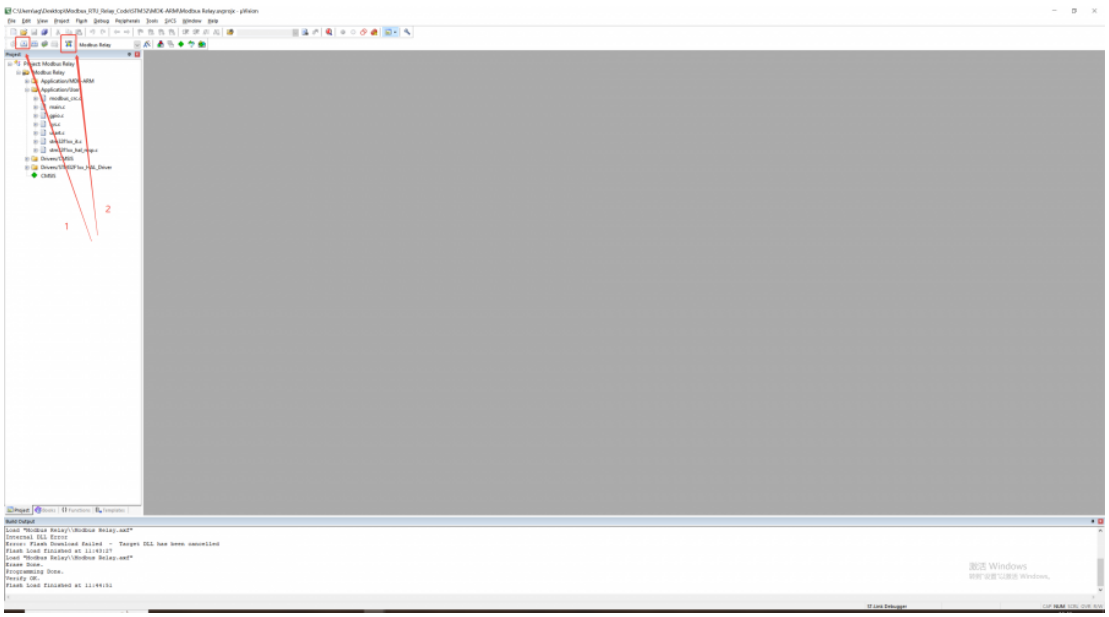
Note: The STM32 demo is based on the NUCLEO-F103RB and RS485 CAN Shield module.

1. Download Demo (https://files.waveshare.com/wiki/Modbus-RTU-Relay-32CH/Modbus_RTU_Relay_32CH_Code.zip), find the STM32 project file Modbus Relay.uvprojx in the path Modbus_RTU_Relay_32CH_Code\STM32\MDK-ARM, and double-click to open the STM32 project file. Note that you should ensure Keil5 software is installed on your computer before using it.



(/wiki/File:900px-Modbus-RTU-Relay-stm32-1.png)

2. Connect the STM32 to a computer via the STM32 download and debug probe. Compile and download the program to the development board.

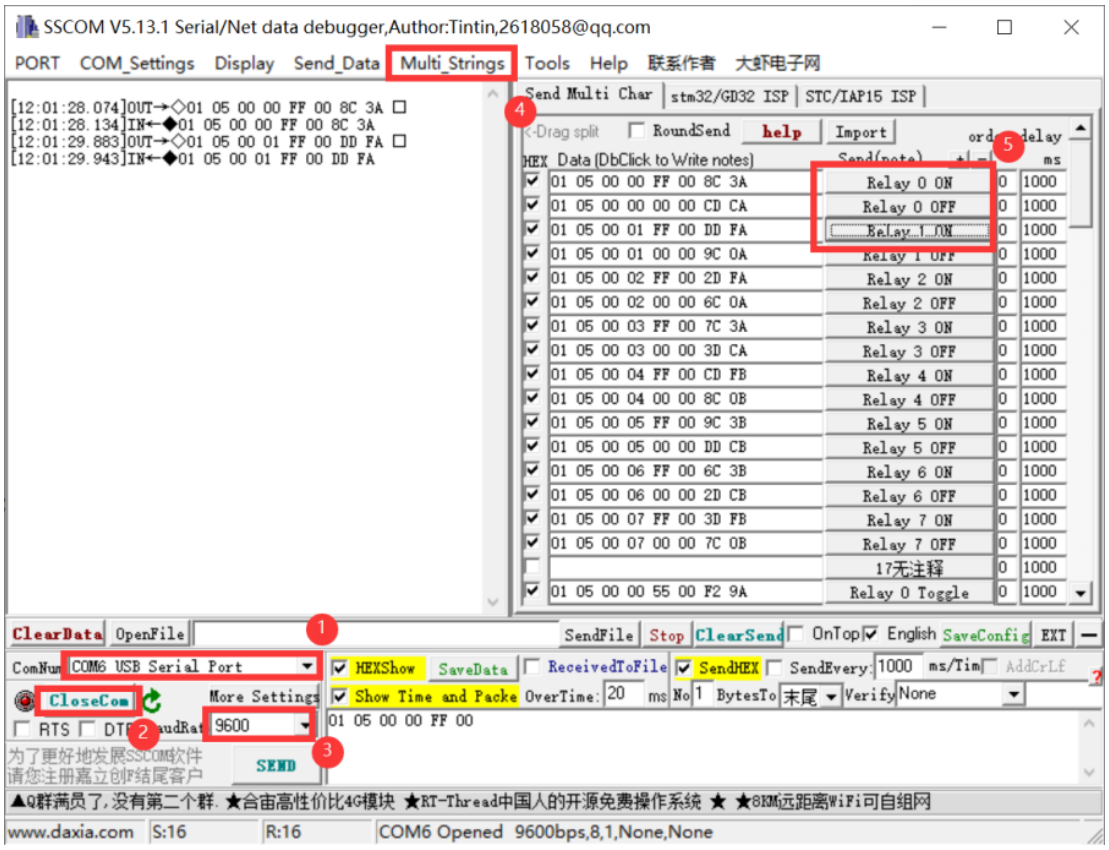


(/wiki/File:900px-Modbus-RTU-Relay-stm32-2.png)

3. Install the RS485 CAN Shield module on the STM32. Connect the RS485_A on the RS485 CAN Shield module to the RS485_A on the Modbus RTU Relay via a wire, and connect the RS485_B on the RS485 CAN Shield module to the RS485_B on the Modbus RTU Relay via a wire. Then

power on the Modbus RTU Relay and the STM32 sequentially.

4. After powering on, the serial port will output the transmitted commands, which can be observed using a serial port assistant. Meanwhile, the relays will sequentially turn on (e.g., 1→2→3→4). Once all relays are fully activated, they will then sequentially turn off (e.g., 1→2→3→4). You can observe whether the relay is normally engaged through the LED indicator light.

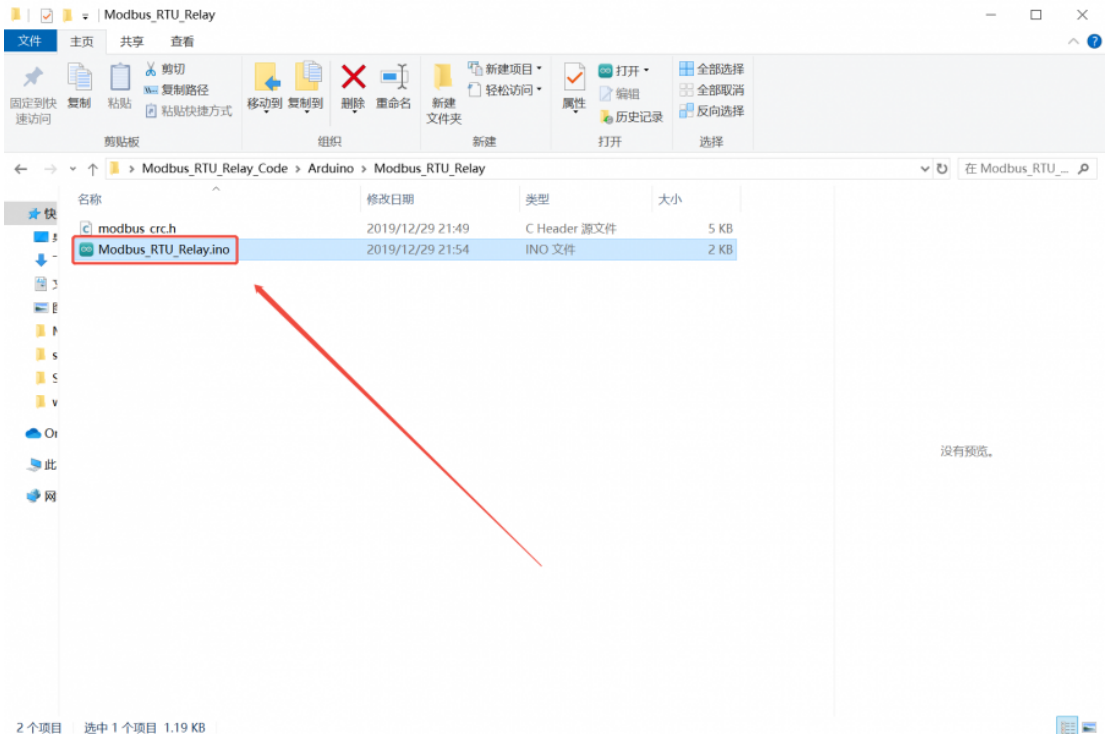


(/wiki/File:Modbus-RTU-Relay-2-1.png)

Arduino

Note: The Arduino demo is based on the UNO PLUS and RS485 CAN Shield module.

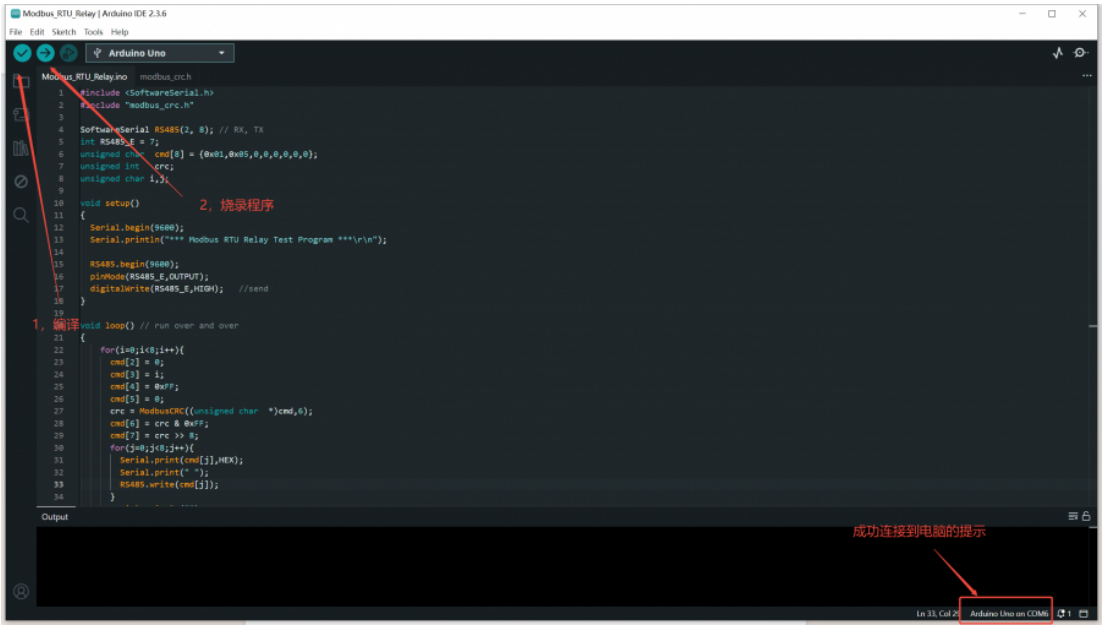
1. Download Demo (https://files.waveshare.com/wiki/Modbus-RTU-Relay-32CH/Modbus_RTU_Relay_32CH_Code.zip), find the Arduino project file Modbus_RTU_Relay.ino in the path Modbus_RTU_Relay_32CH_Code\Arduino\Modbus_RTU_Relay, and double-click to open the Arduino project file. Note that you should ensure Arduino IDE software is installed on your computer before using it.



(/wiki/File:Modbus-RTU-Relay-arduino-1.png)

2. Connect the Arduino to the computer via a USB cable. In the Arduino IDE software, select the Arduino board model under Tools->Board. Choose the COM port that the Arduino is connected to under Tools->Port.

3. After seeing the prompt to connect to the computer in the lower right corner, click to compile and flash the program, and wait for the flashing to complete.



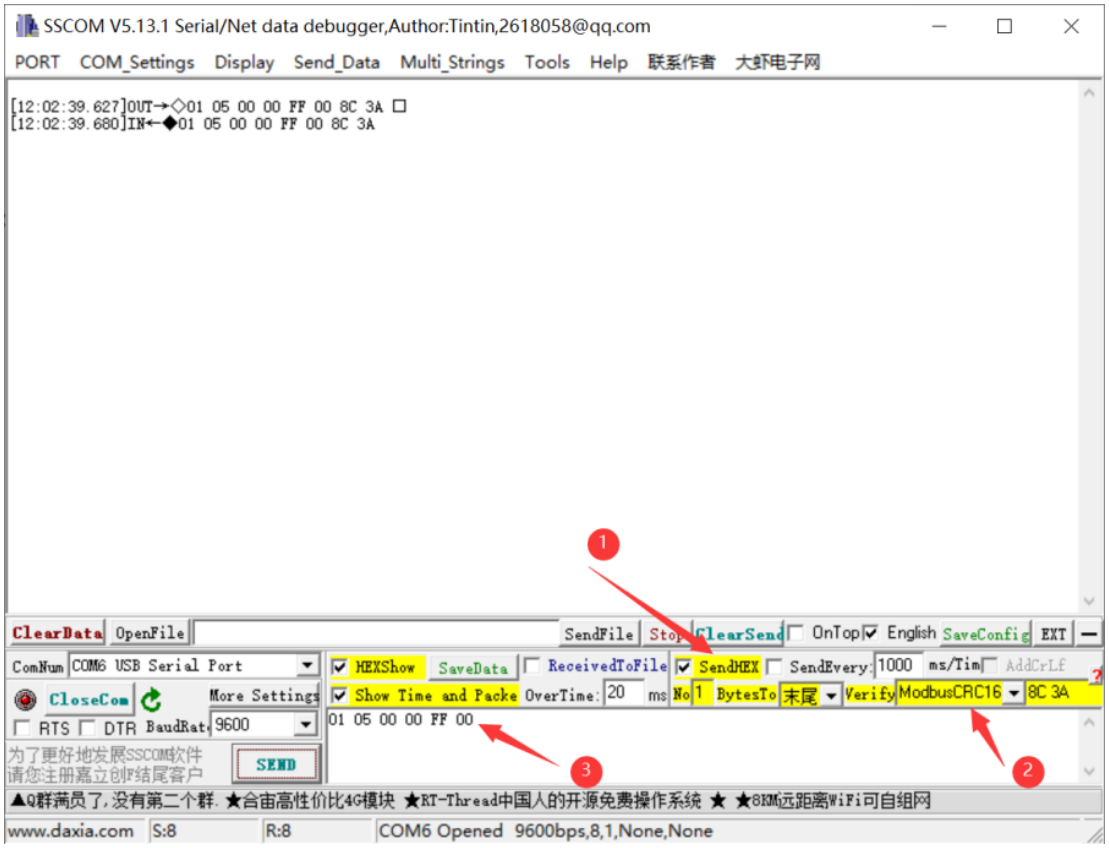
(/wiki/File:Modbus-RTU-Relay-arduino-2.png)

4. Install the RS485 CAN Shield module on the Arduino. Connect the RS485_A on the RS485 CAN Shield module to the RS485_A on the Modbus RTU Relay via a wire, and connect the RS485_B on the RS485 CAN Shield module to the RS485_B on the Modbus RTU Relay via a wire. Then power on the Modbus RTU Relay and the Arduino sequentially.

5. After powering on, the serial port will output the transmitted commands, which can be

observed using a serial port assistant. Meanwhile, the relays will sequentially turn on (e.g., 1→2→3→4). Once all relays are fully activated, they will then sequentially turn off (e.g., 1→2→3→4). You can observe whether the relay is normally engaged through the LED indicator light.

Note: The data in the figure below is in ASCII code containing spaces and line breaks, and is not the actual data sent.

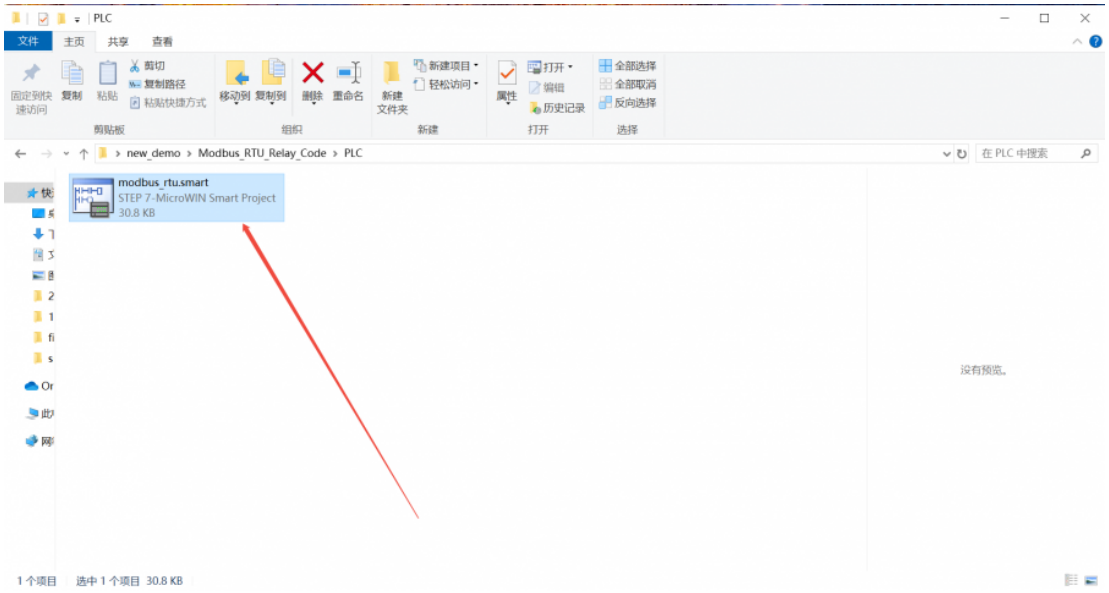


(/wiki/File:Modbus-RTU-Relay-3-1.png)

PLC

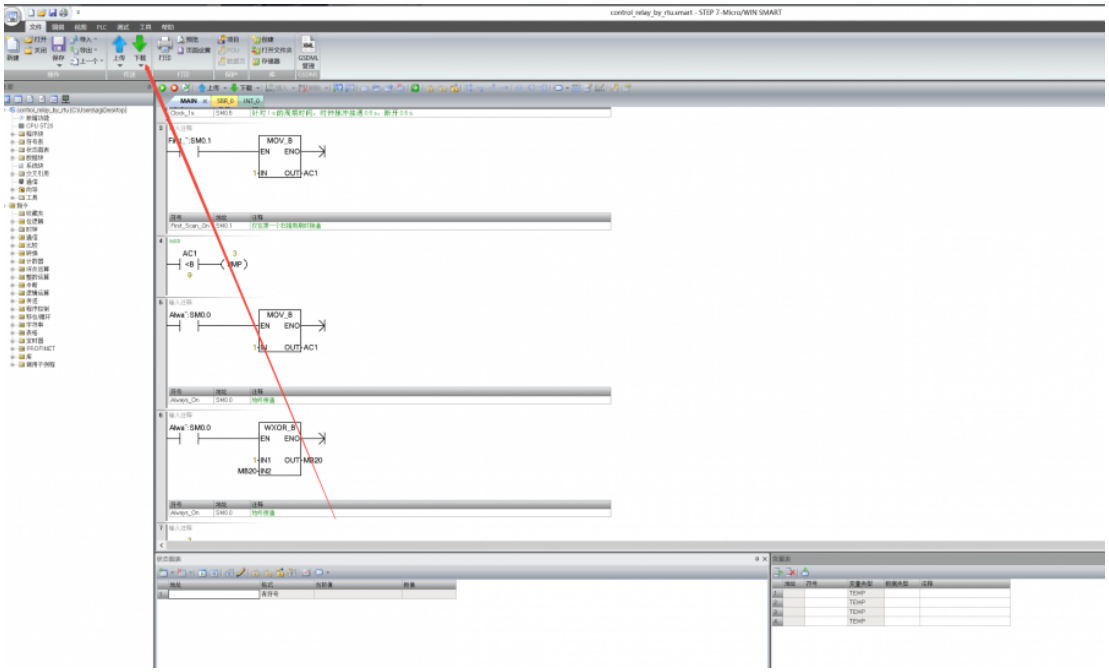
Note: The PLC demo is based on SIMATIC S7-200 SMART.

1. Download Demo (https://files.waveshare.com/wiki/Modbus-RTU-Relay-32CH/Modbus_RTU_Relay_32CH_Code.zip), find the PLC project file modbus_rtu.smart in the path Modbus_RTU_Relay_32CH_Code\PLC, and double-click to open the PLC project file. Note that you should ensure STEP 7-MicroWIN SMART software is installed on your computer before using it.



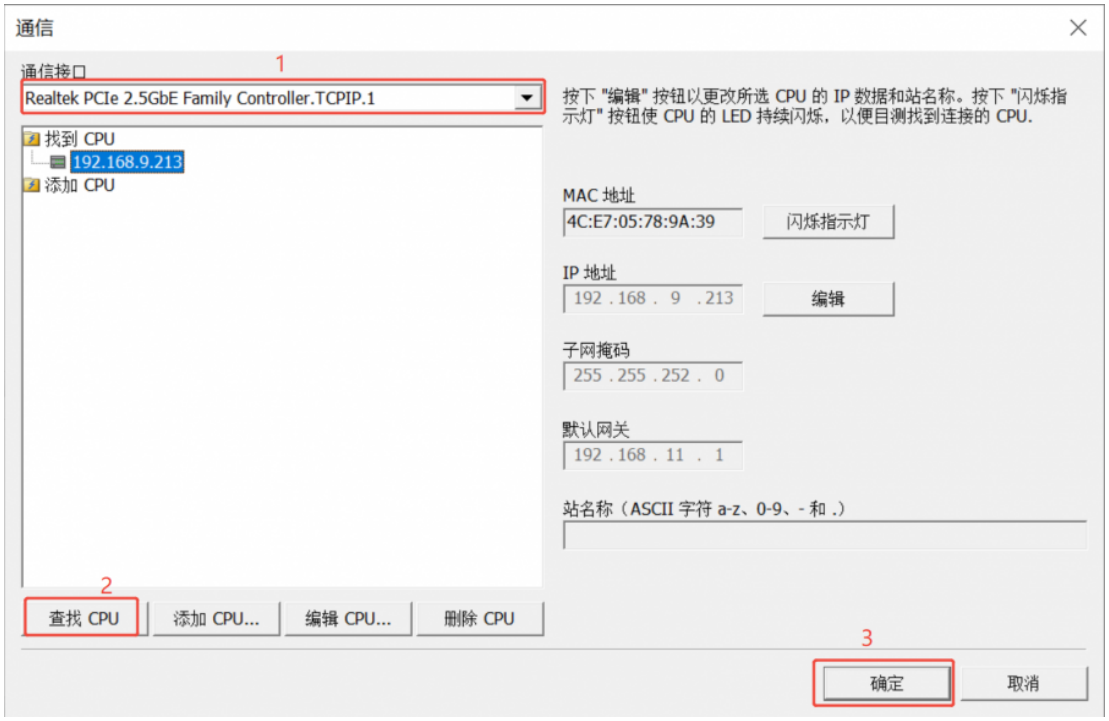
(/wiki/File:Modbus-RTU-Relay-plc-1.png)

2. Connect the PLC to the computer via a network cable. Click to download



(/wiki/File:Modbus-RTU-Relay-plc-2.png)

3. Select the communication interface in the communication popup, find your device, and click Confirm



(/wiki/File:Modbus-RTU-Relay-plc-3.png)

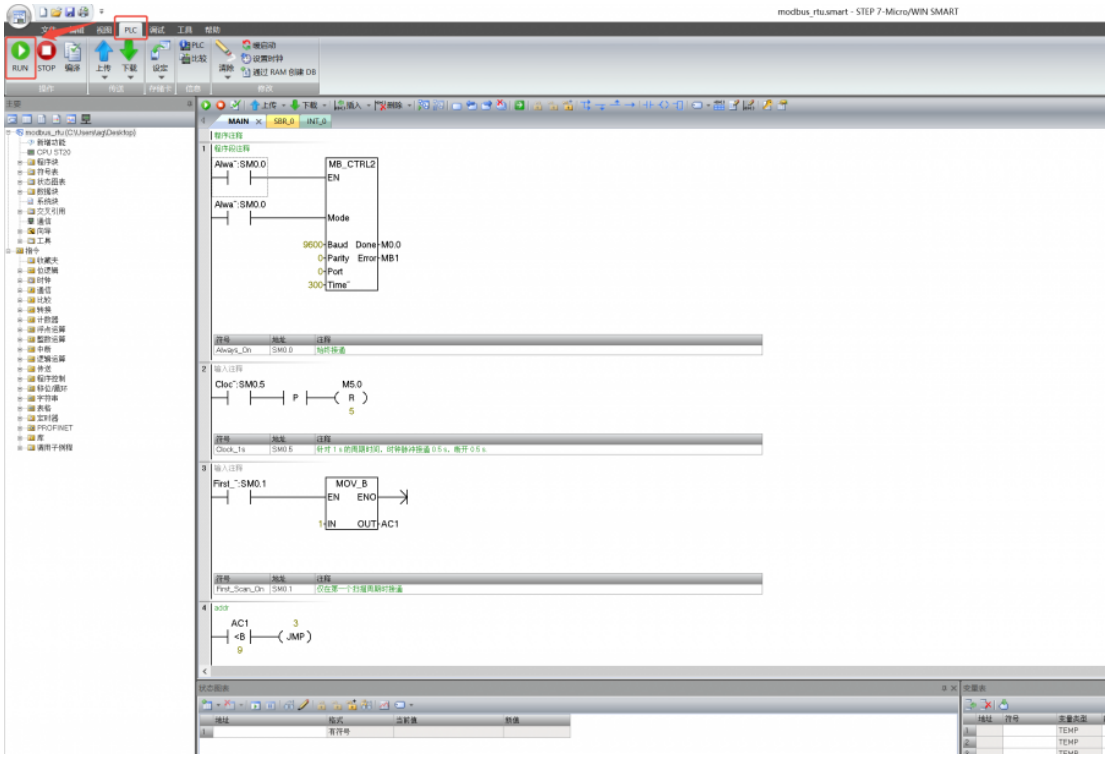
4. In the download pop-up window, check the boxes for program blocks, data blocks, and system blocks, and then click Download.



(/wiki/File:Modbus-RTU-Relay-plc-4.png)

5. Connect the 485_A on the PLC module to RS485_A on the Modbus RTU Relay with a wire, and connect the 485_B on the PLC module to RS485_B on the Modbus RTU Relay with a wire. Then power on the Modbus RTU Relay.

6. After powering on the Modbus RTU Relay, in the STEP 7-MicroWIN SMART software, find the "PLC" tab in the upper menu bar, switch to this tab, and click the green "RUN" button in the lower "Operation" area to perform the operation to make the PLC enter the running state



(/wiki/File:Modbus-RTU-Relay-plc-5.png)

7. After the PLC runs, the relays will sequentially turn on (e.g., 1→2→3→4). Once all relays are fully activated, they will then sequentially turn off (e.g., 1→2→3→4). You can observe whether the relay is normally engaged through the LED indicator light.

Development Protocol V2

Function Code Introduction

Function Code	Description	Note
01	Read coil status	Read relay status
03	Read holding register	Read the address and version
05	Write single coil	Write single relay
06	Write single register	Set the baud rate and address
0F	Write multiple coils	Write all relays

Register Address Introduction

Address (HEX)	Address storage content	Register value	Permission	Modbus Function Code
0x0000 ..... 0x001F	Channels 1~32 relay addresses	0xFF00: relay on 0x0000: relay off 0x5500: relay toggle	Read/Write	0x01, 0x05, 0x0F
0x00FF	Control all relays	0xFF00: all relays on 0x0000: all relays off 0x5500: all relays toggle	Write	0x05
0x0100 ..... 0x011F	Channels 1~32 relays toggle	0xFF00: relay toggle 0x0000: relay unchanged	Write	0x05, 0x0F
0x01FF	All relays toggle	0xFF00: all relays toggle 0x0000: all relays unchanged	Write	0x05
0x0200 ..... 0x021F	Channels 1~32 relays flash on	Delay time: data*100ms Value: 0x0007, delay time: 7*100MS = 700MS	Write	0x05
0x0400 ..... 0x041F	Channels 1~32 relays flash off	Delay time: data*100ms Value: 0x0007, delay time: 7*100MS = 700MS	Write	0x05
4x2000	UART Parameter	The high eight bits indicate the parity mode: 0x00~0x02 The low eight bits indicate the baud rate mode: 0x00~0x07	Read/Write	0x03, 0x06
4x4000	Device Address	Directly store Modbus address Device address: 0x0001-0x00FF	Read/Write	0x03, 0x06
4x8000	Software Version	Converting to decimal and then shifting the decimal point two places to the left will represent the software version 0x0064 = 100 = V1.00	Read	0x03

Modbus RTU Command Introduction

Control Single Relay

Send code: 01 05 00 00 FF 00 8C 3A

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
05	05 Command	Relay control
00 00	Address	The register address of the relay to be controlled, 0x0000-0x001F
FF 00	Command	0xFF00: relay on; 0x0000: relay off; 0x5500: relay toggle
8C 3A	CRC16	The CRC16 checksum of the first 6 bytes of data

Return code: 01 05 00 00 FF 00 8C 3A

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
05	05 Command	Relay control
00 00	Address	The register address of the relay to be controlled, 0x0000-0x001F
FF 00	Command	0xFF00: relay on; 0x0000: relay off; 0x5500: relay toggle
8C 3A	CRC16	The CRC16 checksum of the first 6 bytes of data

For example: [Address 1 device]:

```
Relay 0 on: 01 05 00 00 FF 00 8C 3A
Relay 0 off: 01 05 00 00 00 00 CD CA
Relay 1 on: 01 05 00 01 FF 00 DD FA
Relay 1 off: 01 05 00 01 00 00 9C 0A
Relay 2 on: 01 05 00 02 FF 00 2D FA
Relay 2 off: 01 05 00 02 00 00 6C 0A
Relay 3 on: 01 05 00 03 FF 00 7C 3A
Relay 3 off: 01 05 00 03 00 00 3D CA
Relay 0 toggle: 01 05 00 00 55 00 F2 9A
Relay 1 toggle: 01 05 00 01 55 00 A3 5A
Relay 2 toggle: 01 05 00 02 55 00 53 5A
Relay 3 toggle: 01 05 00 03 55 00 02 9A
```

Control All Relays

Send code: 01 05 00 FF FF 00 BC 0A

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
05	05 Command	Relay control
00 FF	Address	Fixed 0x00FF
FF 00	Command	0xFF00: relay on; 0x0000: relay off; 0x5500: relay toggle
BC 0A	CRC16	The CRC16 checksum of the first 6 bytes of data

Return code: 01 05 00 FF FF 00 BC 0A

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
05	05 Command	Relay control
00 FF	Address	Fixed 0x00FF
FF 00	Command	0xFF00: relay on; 0x0000: relay off; 0x5500: relay toggle
BC 0A	CRC16	The CRC16 checksum of the first 6 bytes of data

For example: [Address 1 device]:

```
All relays on: 01 05 00 FF FF 00 BC 0A
All relays off: 01 05 00 FF 00 00 FD FA
All relays toggle: 01 05 00 FF 55 00 C2 AA
```

Read Relay Status

Send code: 01 01 00 00 00 20 3D D2

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
01	01 Command	Query relay status
00 00	Relay Start Address	The register address of the relay, 0x0000 - 0x001F
00 20	Relay Number	The number of relays to be read, which must not exceed the maximum number of relays
3D D2	CRC16	The CRC16 checksum of the first 6 bytes of data

Receive code: 01 01 04 00 00 00 00 FB D1

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
01	01 Command	Query relay status
04	Byte Number	The number of all bytes of the returned status information
00 00 00 00	Query status	Received relay status The relay state is in small-endian format, with low bytes first and high bytes last Bit0: the first relay status; Bit1: the second relay status; And so on, with the idle high bit being zero
FB D1	CRC16	The CRC16 checksum of the first 7 bytes of data

For example: [Address 1 device]

```
Send: 01 01 00 00 00 20 3D D2          //Query all relays
Receive: 01 01 04 00 00 00 00 FB D1    //all relays off
Send: 01 01 00 01 00 03 2D CB          //Query relays 1, 2, 3 status
Receive: 01 01 01 05 91 8B             //Relays 1 and 3 are on, relay 2 is off
Send: 01 01 00 04 00 0C 7D CE          //Query relays 4-15
Receive: 01 01 02 0F 00 BC 0C          //Relays 4-7 are on, relays 8-15 are off
```

Write Relay Status

Send code: 01 0F 00 00 00 20 04 FF FF FF FF C5 1C

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
0F	0F Command	Write relay status
00 00	Relay Start Address	The register address of the relay to be controlled, 0x0000 - 0x001F
00 20	Relay Number	The number of relays to be operated, which must not exceed the maximum number of relays
04	Byte Number	The byte number of the status
FF FF FF FF	Relay status	The relay state is in small-endian format, with low bytes first and high bytes last Bit0: the first relay status; Bit1: the second relay status; And so on, with the idle high bit being zero
C5 1C	CRC16	The CRC16 checksum of the first 11 bytes of data

Receive code: 01 0F 00 00 00 20 54 13

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
0F	0F Command	Control all registers
00 00	Relay Start Address	The register address of the relay to be controlled, 0x0000 - 0x001F
00 20	Relay Number	The number of relays to be operated
54 13	CRC16	The CRC16 checksum of the first 6 bytes of data

For example: [Address 1 device]

```
All relays on: 01 0F 00 00 00 20 04 FF FF FF FF C5 1C
All relays off: 01 0F 00 00 00 20 04 00 00 00 00 C4 88
0-1 on; 3-31 off: 01 0F 00 00 00 20 04 03 00 00 00 C4 CC
```

Relay Flash ON/OFF Command

Send code: 01 05 02 00 00 07 8D B0

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
05	05 Command	Single control command
02	Flash on flash off	02: flash on, 04: flash off
00	Relay Address	The address of the relay to be controlled, 0x00~0x1F
00 07	Delay Time	The delay time: data*100ms Value: 0x0007, delay time: 7*100MS = 700MS The maximum setting for the flash-on flash-off time is 0x7FFF
8D B0	CRC16	The CRC16 checksum of the first 6 bytes of data

Receive code: 01 05 02 00 00 07 8D B0

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
05	05 Command	Single control command
02	Flash on flash off	02: flash on, 04: flash off
00	Relay Address	The address of the relay to be controlled, 0x00~0x1F
00 07	Delay Time	The delay time: data*100ms Value: 0x0007, delay time: 7*100MS = 700MS The maximum setting for the flash-on flash-off time is 0x7FFF
8D B0	CRC16	The CRC16 checksum of the first 6 bytes of data

For example: [Address 1 device]

```
Relay 0 flash on: 01 05 02 00 00 07 8D B0   //700MS = 7*100MS = 700MS
Relay 1 flash on: 01 05 02 01 00 08 9C 74   //800MS
Relay 0 flash off: 01 05 04 00 00 05 0C F9  //500MS
Relay 1 flash off: 01 05 04 01 00 06 1D 38  //600MS
```

Set Baudrate Command

Send code: 01 06 20 00 00 05 42 09

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
06	06 command	Set the baud rate and device address
20 00	Baud rate register	0x2000: set the baud rate
00	Parity Method	0x00: no parity, 0x01: even parity; 0x02: odd parity
05	Baud Rate Value	Correspondence of baud rate values 0x00: 4800 0x01: 9600 0x02: 19200 0x03: 38400 0x04: 57600 0x05: 115200 0x06: 128000 0x07: 256000
42 09	CRC16	The CRC16 checksum of the first 6 bytes of data

Receive code: 01 06 20 00 00 05 42 09

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
06	06 command	Set the baud rate and device address
20 00	Baud rate register	0x2000: set the baud rate
00	Parity Method	0x00: no parity, 0x01: odd parity; 0x02: even parity
05	Baud Rate	Correspondence of baud rate values 0x00: 4800 0x01: 9600 0x02: 19200 0x03: 38400 0x04: 57600 0x05: 115200 0x06: 128000 0x07: 256000
42 09	CRC16	The CRC16 checksum of the first 6 bytes of data

For example: [Address 1 device]

```
Set the baud rate as 4800: 01 06 20 00 00 00 82 0A
Set the baud rate as 9600: 01 06 20 00 00 01 43 CA
Set the baud rate as 115200: 01 06 20 00 00 05 42 09
```

Set Device Address Command

Send code: 01 06 40 00 00 01 5D CA

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
06	06 command	Set the baud rate and device address
40 00	Device address register	0x4000: set the device address
00 01	Device Address	Set the device address, 0x0001-0x00FF
5D CA	CRC16	The CRC16 checksum of the first 6 bytes of data

Receive code: 01 06 40 00 00 01 5D CA

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
06	06 command	Set the baud rate and device address
40 00	Device address register	0x4000: set the device address
00 01	Device Address	Set the device address, 0x0001-0x00FF
5D CA	CRC16	The CRC16 checksum of the first 6 bytes of data

For example: [Address 1 device]

```
Set the device address as 0x01: 00 06 40 00 00 01 5C 1B
Set the device address as 0x02: 00 06 40 00 00 02 1C 1A
Set the device address as 0x03: 00 06 40 00 00 03 DD DA
```

Read Device Address Command

Send code: 00 03 40 00 00 01 90 1B

Field	Description	Note
00	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
03	03 Command	Read the device address
40 00	Device address register	0x4000: read the device address
00 01	Byte Number	Fixed 0x0001
90 1B	CRC16	The CRC16 checksum of the first 6 bytes of data

Receive code: 00 03 02 00 01 44 44

Field	Description	Note
00	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
03	03 Command	Read the software version and device address
02	Byte Number	The number of bytes returned
00 01	Device Address	Set the device address, 0x0001-0x00FF
44 44	CRC16	The CRC16 checksum of the first 5 bytes of data

For example: [Address 2 device]

```
Send: 00 03 40 00 00 01 90 1B
Receive: 00 03 02 00 02 04 45     //Address: 0x02
```

Read Software Version Command

Send code: 01 03 80 00 00 01 AD CA

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
03	03 Command	Read the software version and device address
80 00	Software version register	0x4000: read the device address, 0x8000: read software version
00 01	Byte Number	Fixed 0x0001
AD CA	CRC16	The CRC16 checksum of the first 6 bytes of data

Receive code: 01 03 02 01 2C B8 09

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
03	03 Command	Read the software version and device address
02	Byte Number	The number of bytes returned
01 2C	Software Version	Converting to decimal and then shifting the decimal point two places to the left will represent the software version 0x012C = 300 = V3.00
B8 09	CRC16	The CRC16 checksum of the first 5 bytes of data

For example:

```
Send: 01 03 80 00 00 01 AD CA
Receive: 01 03 02 01 2C B8 09       //0x012C = 300 =V3.00
```

Exception Function Code

When the received command is incorrect or the device is abnormal, an exception response will be returned in the following format:

Return code: 01 85 03 02 91

Field	Description	Note
01	Device Address	0x00 indicates the broadcast address, 0x01-0xFF indicates the device address
85	Exception Function Code	Exception function code = Request function code + 0x80
03	Byte Number	Exception Code
02 91	CRC16	The CRC16 checksum of the first 3 bytes of data

An exception code is a single-byte value that indicates the type of error. Several commonly used exception codes defined by the Modbus protocol:

Exception Code	Name	Description
0x01	Illegal Function	The requested function code is not supported
0x02	Illegal Data Address	The requested data address is incorrect
0x03	Illegal Data Value	The requested data value or operation cannot be executed
0x04	Server Failure	Server equipment failure
0x05	Response	The request has been received and is being processed
0x06	Device Busy	The device is currently busy and cannot perform the requested operation

Resources

Demo

- Demo (https://files.waveshare.com/wiki/Modbus-RTU-Relay-32CH/Modbus_RTU_Relay_32CH_Code.zip)

Software

- Sscom5.13.1_for_Modbus_RTU_Relay_32CH.zip (https://files.waveshare.com/wiki/Modbus-RTU-Relay-32CH/Sscom5.13.1_for_Modbus_RTU_Relay_32CH.zip)
- Modbus Poll software (https://www.modbustools.com/download.html)
- SecureCRT software (https://files.waveshare.com/wiki/common/SecureCRT.7z)

Related resources

- Modbus Protocol Specification (https://www.waveshare.com/wiki/Modbus_Protocol_Specification)
- Modbus Series BootLoader Description (https://www.waveshare.com/wiki/Modbus_Series_BootLoader_Description)
- Protocol V1 (https://www.waveshare.com/wiki/Template:Mobus_RTC_Relay_32CH_Protocol)
- Using Modbus RTU Relay with Home Assistant (https://www.waveshare.com/wiki/Using_Modbus_RTU_Relay_with_Homeassistant)

FAQ

Question: Where can I download the driver for my device?

Answer:

The device does not require an additional driver download. You can control the device relay's on/off or query the relay status by sending Modbus commands through the device's 485 interface.

Question: Can relay components use the module's own power supply to output voltage/current to the normally open/normally closed terminals?

Answer:

No

Question: Can the slave address be set by a DIP switch?

Answer:

There is no DIP switch, it can only be set through commands

Question: What should I do if I forget the communication parameters of the 485?

Answer:

Please refer to Modbus Series BootLoader Description (https://www.waveshare.com/wiki/Modbus_Series_BootLoader_Description) to enter the boot mode, then you can select option 3 to read communication parameters or select option 2 to reset communication parameters

Question: What should I do if the SecureCRT session window is garbled when entering boot mode?

Answer:

Check if the 485 bus is reversed. If not, connect a 120 Ω terminal resistor across the A and B

cables

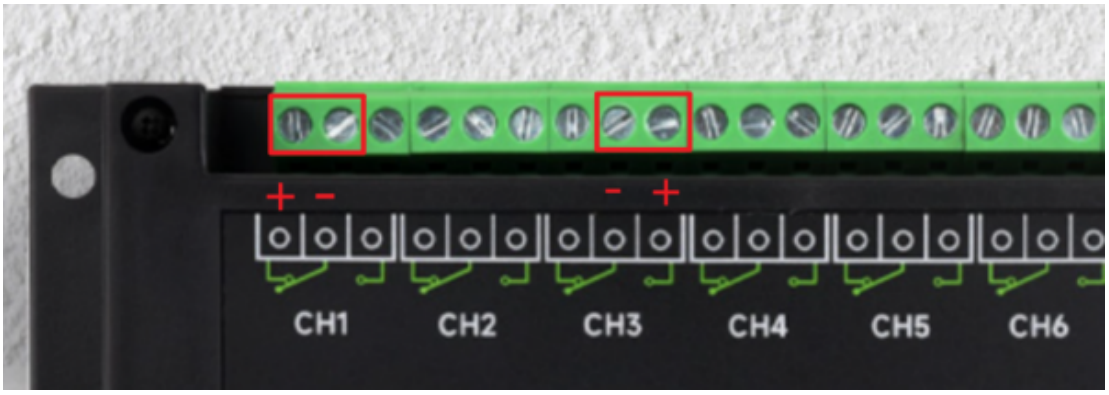
Question: If you want to make the circuit remain in the default connected/ disconnected state, how should the wiring be done?

Answer:

The default connection state, also known as the normally closed state, can be seen on Channel 1. The terminals within the red box above Channel 1 are connected by default; when turned on, they become disconnected.

The default disconnection state, also known as the normally open state, wiring can be seen on Channel 3. The terminals within the red box above Channel 3 are disconnected by default; when turned on, they become connected.

Note: The positive and negative wiring method shown in the diagram is for reference only. Please adjust to the appropriate positive and negative terminal connections according to the specific situation during actual use.



(/wiki/File:600px-Modbus_POE_ETH_Relay_Open_close_wiring_diagram.png)

Question: What is the power consumption of the device?

Answer:

Under 12V power supply, the reference power consumption when in standby mode (all relays in the off state): 0.3W, the reference power consumption when relays are in the on state: 11.7W.

Please refer to actual conditions for specific power consumption

Question: How to power the device?

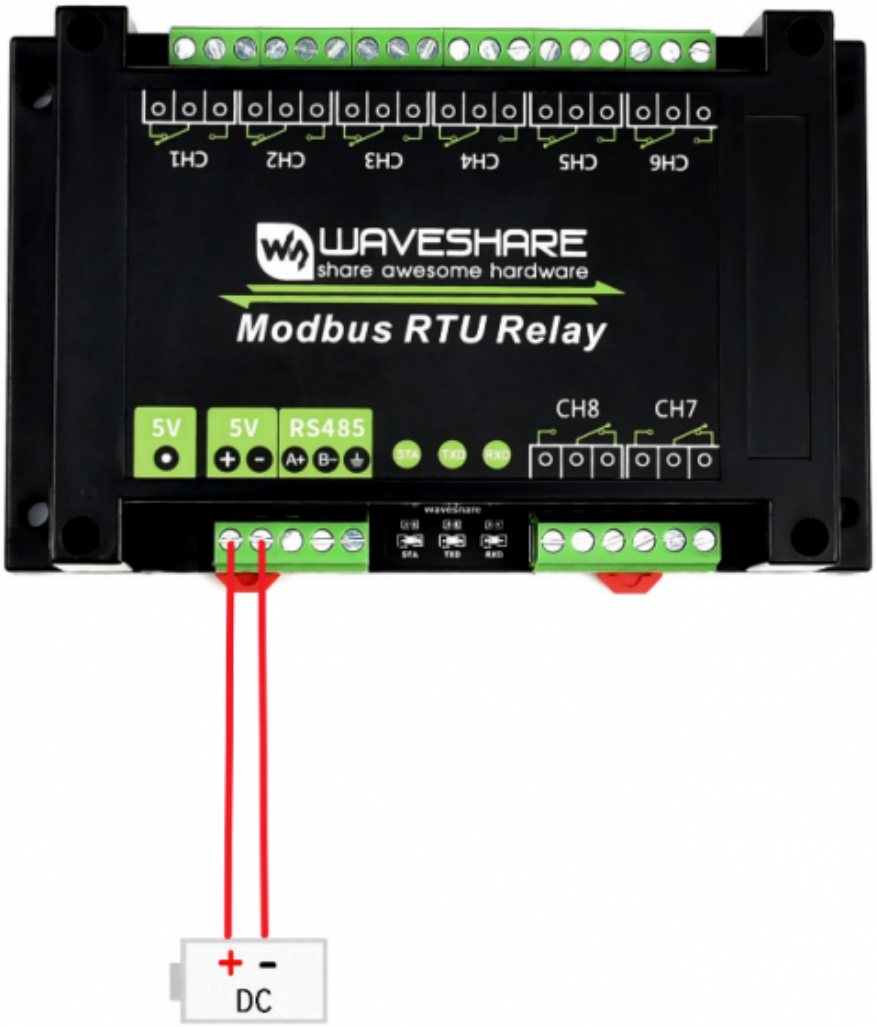
Answer:

Power via circular DC interface or via screw terminals, choose one of the two DC power supply methods.

Warning: To ensure personal and equipment safety, do not use both wiring methods for power supply at the same time.

Method 1: 

(/wiki/File:600px-Supplying_power_to_the_relay_Method1.png)

Method 2: 

(/wiki/File:600px-Supplying_power_to_the_relay_Method2.png)

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve the

Submit Now (https://service.waveshare.com/)

issue.

Working Time: 9 AM - 6 PM GMT+8

(Monday to Friday)

Retrieved from "https://www.waveshare.com/w/index.php?title=Modbus_RTU_Relay_32CH&oldid=107666 (https://www.waveshare.com/w/index.php?title=Modbus_RTU_Relay_32CH&oldid=107666)"